

06 — Role, Tech Stack and Node.js Backend Q&A

How to answer technical questions: 1) Simple definition → 2) How it works (1–2 lines) → 3) Where I used it in my project. Step 3 is what makes you sound experienced.

PART 1 — Explaining your role and tech stack

"What is your role in your current company?"

"I'm a MERN stack developer with a backend focus. I own the backend systems end to end: I take requirements from business users, design APIs and database queries, write the Node.js code, integrate third-party APIs, deploy to IIS or Cloudways, and support the apps in production. I also build the React screens when needed."

"Explain your tech stack."

"My core stack is Node.js with Express for the backend. For databases I use MongoDB with Mongoose, and SQL databases — SQL Server, PostgreSQL and MySQL. I use Redis for caching and as a queue broker with BullMQ. For real-time I use Socket.IO. On the frontend I use React with Redux Toolkit and Tailwind. For deployment I've used Docker, PM2, NGINX, IIS with iisnode, AWS Lambda, EC2 and S3, with CI/CD on GitHub Actions and Bitbucket."

"What does your typical day look like?"

"Morning: check production logs and any issues from users. Then requirement discussion if a new feature is coming. Then development — usually backend first: SQL query, repository, service, controller, route and validation, tested in Postman. Then the frontend screen. Then build, deploy, check health endpoints and verify with real users. I commit with clear messages and update documentation."

"How do you structure a Node.js project?"

"Layered structure: routes (only HTTP mapping) → validations (Joi schemas) → controllers (thin, handle request/response) → services (business logic) → repositories (database queries). Plus middlewares (auth, error handler, rate limiter), config (env validation), utils (ApiError, ApiResponse, logger). Each feature becomes 5 small files, easy to test and easy for others to find."

```
src/  
  config/      env.js, db.js, redis.js  
  routes/      appointment.routes.js  
  validations/ appointment.validation.js  
  controllers/ appointment.controller.js  
  services/     appointment.service.js  
  repositories/ appointment.repository.js  
  middlewares/  auth.js, errorHandler.js, rateLimit.js  
  utils/        ApiError.js, ApiResponse.js, logger.js  
  app.js, server.js
```

PART 2 — JavaScript core (asked in almost every Node interview)

1. var vs let vs const? `var` is function-scoped and hoisted with `undefined`. `let` and `const` are block-scoped and in the "temporal dead zone" until declared. `const` cannot be reassigned, but object contents can change.

2. What is hoisting? Declarations move to the top of their scope before execution. Function declarations are fully hoisted; `var` is hoisted as `undefined`; `let/const` are hoisted but not initialised.

3. What is a closure? A function that remembers variables from its outer scope even after the outer function has finished.

```
function counter() { let c = 0; return () => ++c; }
const next = counter(); next(); // 1
next(); // 2
```

Used for: private variables, memoisation, middleware factories like `authorize(role)`.

4. == vs ===? `==` converts types before comparing (`'5' == 5` is true). `===` compares value and type. Always use `===`.

5. Promise, async/await? A Promise represents a future value: pending → fulfilled or rejected. `async/await` is cleaner syntax over promises. Errors are handled with `try/catch`.

6. Promise.all vs allSettled vs race vs any? - `all`: waits for all; fails fast if one rejects. - `allSettled`: waits for all, gives status of each — I use it when one failed API should not fail the whole report. - `race`: first to settle (useful for timeouts). - `any`: first to fulfil.

7. Event loop, microtask vs macrotask? Call stack runs sync code. Then all **microtasks** run (promise callbacks, `process.nextTick` first in Node). Then one **macrotask** (`setTimeout`, I/O). Output of:

```
console.log(1); setTimeout(()=>console.log(2)); Promise.resolve().then(()=>console.log(3));
console.log(4);
// 1 4 3 2
```

8. this keyword and arrow functions? `this` depends on how a function is called. Arrow functions don't have their own `this`; they take it from the outer scope.

9. call, apply, bind? All set `this`. `call(obj, a, b)` runs now with arguments; `apply(obj, [a, b])` runs now with an array; `bind(obj)` returns a new function for later.

10. Shallow vs deep copy? Spread `{...obj}` copies one level. Deep copy: `structuredClone(obj)`.

11. Debounce vs throttle? Debounce runs after the user stops for X ms (search box). Throttle runs at most once every X ms (scroll).

12. map vs forEach vs filter vs reduce? `map` returns a new transformed array; `forEach` returns nothing; `filter` returns matching items; `reduce` builds one value (sum, grouped object).

13. What is prototype inheritance? Every object has a hidden link to a prototype object. Property lookup goes up this chain. `class` syntax is built on prototypes.

14. Spread vs rest? Same `...` syntax. Spread expands (`[...arr]`), rest collects (`function f(...args)`).

PART 3 — Node.js

15. What is Node.js? A JavaScript runtime built on Chrome's V8 engine. It's single-threaded with a non-blocking, event-driven I/O model, which makes it great for I/O-heavy apps like APIs and real-time systems.

16. How does Node handle many requests with one thread?

"The main thread runs JavaScript. I/O like database calls, file reads and network calls is handed to the OS or libuv's thread pool. When the work finishes, a callback is queued and the event loop runs it. So the thread is never waiting idle. But CPU-heavy code blocks everything." *My example:* "In my reporting app, a heavy synchronous SQLite query blocked the event loop and froze the app. I added a date-range guard and moved index building to a worker thread."

17. Event loop phases? Timers (setTimeout/setInterval) → pending callbacks → poll (I/O) → check (setImmediate) → close callbacks. `process.nextTick` and promise microtasks run between phases.

18. setImmediate vs setTimeout(0) vs process.nextTick? `nextTick` runs right after the current operation, before other microtasks. `setImmediate` runs in the check phase after I/O. `setTimeout(0)` runs in the timers phase. Inside an I/O callback, `setImmediate` always runs before `setTimeout(0)`.

19. What is libuv? The C library behind Node's event loop and thread pool (default 4 threads) used for file system, DNS, crypto and compression.

20. Worker threads vs cluster vs child process? - **Worker threads:** run CPU-heavy JS in parallel inside one process (shared memory possible). - **Cluster:** runs multiple Node processes on the same port to use all CPU cores. PM2 cluster mode does this. - **Child process:** run another program or script (`spawn` , `exec` , `fork`).

21. What are streams? Types? Process data piece by piece instead of loading everything into memory. Types: Readable, Writable, Duplex, Transform. Use `pipeline()` for safe piping with error handling.

My example: "I stream call recordings from the dialer to the browser with HTTP Range support, and I use database cursors to sync large tables in batches."

22. What is a Buffer? Raw binary data in memory outside the V8 heap, used for files, images and network data.

23. CommonJS vs ES Modules? CommonJS: `require` / `module.exports` , loaded synchronously. ESM: `import` / `export` , static and async, needs `"type": "module"` . In ESM there's no `__dirname` — use `import.meta.url` . My Atiya backends use ESM.

24. What is middleware? (Node/Express) A function (`req` , `res` , `next`) that runs between the request and the response — for logging, auth, validation, parsing or error handling. Calling `next()` passes control forward.

25. How do you handle errors in Node?

" try/catch with async/await, a central Express error-handler middleware, a custom `ApiError` class with status codes, and process-level handlers for `unhandledRejection` and `uncaughtException` that log the error. For real bugs the process exits and PM2 or IIS restarts it."

26. How do you do graceful shutdown? On `SIGTERM` / `SIGINT` : stop accepting new connections (`server.close()`), finish running requests, close DB and Redis connections, then exit. Add a 10-second

forced-exit timer.

27. How do you find a memory leak? Watch memory in PM2 or monitoring. Take heap snapshots with `--inspect` and Chrome DevTools and compare. Common causes: global arrays or caches that keep growing, event listeners not removed, unclosed timers. Fix with bounded LRU caches and cleanup.

28. How do you improve Node.js performance?

"Use indexes and optimised queries, cache hot data in Redis, paginate, stream large data, avoid blocking the event loop, use cluster mode or more instances, compression, connection pooling, and move heavy work to background queues."

29. What is package-lock.json? It locks exact dependency versions so every install is the same. Use `npm ci` in CI/CD.

30. dependencies vs devDependencies? `dependencies` are needed at runtime (express). `devDependencies` only for development (jest, eslint). In production: `npm ci --omit=dev`.

31. How do you manage environment config? `.env` files per environment, loaded with dotenv, validated at startup with a schema (envalid / Joi) so the app fails fast if a variable is missing. Never commit real secrets.

PART 4 — Express.js

32. What is Express? A minimal web framework for Node that provides routing, middleware and helpers for request/response.

33. Middleware order in your apps? `helmet` → `cors` → `compression` → body parser with size limit → request ID → logger → rate limiter → auth → routes → 404 handler → error handler. Webhook routes are mounted **before** auth because the provider calls them without a token.

34. Error-handling middleware? Has 4 arguments (`err`, `req`, `res`, `next`) and must be registered last.

```
app.use((err, req, res, next) => {
  const status = err.statusCode || 500;
  logger.error(err);
  res.status(status).json({ success: false, message: status === 500 ? 'Internal error' :
err.message });
});
```

35. Express 4 vs Express 5? Express 5 automatically catches rejected promises in async handlers (no `express-async-errors` needed), `req.query` is read-only, and route path syntax changed.

36. app.use vs app.get? `app.use` mounts middleware for all methods on a path prefix. `app.get` handles only GET on an exact route.

37. req.params vs req.query vs req.body? `params`: from URL path `/users/:id`. `query`: from `?page=2`. `body`: from POST/PUT payload.

38. How do you validate input? Joi schemas through a generic `validate(schema)` middleware for body, query and params. Invalid input returns 400 with clear messages.

39. How do you upload files? `multer` with size limits and file-type checks. Save to a temp folder or memory, then upload to S3 or Cloudinary. (*I used multer + Cloudinary in the WhatsApp platform.*)

40. How do you implement CORS? `cors` middleware with an allowlist of origins. Or avoid CORS by serving the API on the same domain through a reverse proxy.

PART 5 — REST API design

41. What is REST? An architectural style where resources are identified by URLs and manipulated with HTTP methods. It's stateless — every request carries everything needed.

42. HTTP methods? GET (read), POST (create), PUT (replace), PATCH (partial update), DELETE (remove). GET, PUT, DELETE are idempotent; POST is not.

43. Important status codes? 200 OK, 201 Created, 204 No Content, 400 Bad Request, 401 Unauthorized (not logged in), 403 Forbidden (no permission), 404 Not Found, 409 Conflict, 413 Payload Too Large, 422 Validation error, 429 Too Many Requests, 500 Server Error, 502 Bad Gateway, 503 Service Unavailable.

44. PUT vs PATCH? PUT replaces the whole resource. PATCH updates only the given fields.

45. How do you design pagination? Offset pagination (`?page=2&limit=20`) is simple but slow for big tables. **Keyset/cursor pagination** (`WHERE id > lastId ORDER BY id LIMIT 20`) is fast for large data.

My example: "I used keyset pagination to sync tables with 50M+ rows."

46. How do you version APIs? In the URL (`/api/v1/...`) — simplest and most common. Old versions stay until clients move.

47. What is idempotency and why does it matter? Doing the same request many times gives the same result. Important for payments and webhooks.

My example: "Exotel can resend a webhook. I upsert by the message ID, so duplicates don't create two records. For Stripe I use idempotency keys."

48. REST vs GraphQL vs tRPC? REST: multiple endpoints, simple caching. GraphQL: one endpoint, client chooses fields, avoids over-fetching. tRPC: type-safe calls between a TypeScript frontend and backend without a schema — used in TorahAnytime.

49. How do you document APIs? Swagger / OpenAPI served at `/api/docs` (used in the Hospital CRM), plus Postman collections.

50. How do you call a third-party API safely? Timeouts, retry with exponential backoff only for network errors / 5xx / 429 (never 4xx), circuit breaker for repeated failures, logging, and secrets from env. Never let a third-party failure crash the main flow.

PART 6 — Authentication and security

51. Authentication vs authorization? Authentication: who are you (login). Authorization: what are you allowed to do (roles, permissions).

52. How does JWT work? After login the server signs a token with a secret. It has 3 parts: header, payload, signature. The client sends it in `Authorization: Bearer <token>`. The server verifies the signature — no session lookup needed.

53. Access token + refresh token? Short-lived access token (15 min) for API calls, long-lived refresh token (7 days) to get a new access token. Store refresh token in an httpOnly secure cookie and in the DB so it can be revoked.

54. Where to store JWT on the frontend? httpOnly, Secure, SameSite cookie is safest against XSS. localStorage is simpler but vulnerable to XSS.

55. How do you store passwords? Hash with bcrypt (salt + cost factor ~10–12). Never store plain text or use reversible encryption for new systems.

56. How did you implement RBAC?

"Users have roles and permission keys. A middleware like `authorize('report:view')` checks the permission on each route. The same permission keys hide menu items in the frontend. In ViewReports every report has its own permission key."

57. How did you implement OTP two-factor login?

"After password check, I generate a 6-digit OTP, store it in Redis with a 5-minute TTL and an attempt counter, and email it. The user submits it, I compare it in constant time, then issue the JWT."

58. What is OAuth 2.0? A standard for delegated access. The user logs in with Google, Google gives an authorization code, the backend exchanges it for tokens. Used for "Login with Google".

59. Common security practices in Node APIs? - Helmet for security headers - CORS allowlist - Rate limiting (express-rate-limit) - Input validation (Joi / zod) - Parameterised queries (prevent SQL injection) - Sanitise MongoDB queries (prevent NoSQL injection like `{ "$gt": "" }`) - Secrets in env, not in code - HTTPS everywhere - `npm audit` / Dependabot for vulnerable packages

60. SQL injection — how to prevent? Never build SQL by joining strings with user input. Use parameterised queries: `request.input('id', sql.Int, id)` in mssql or `$1` in pg.

61. XSS and CSRF? XSS: attacker injects script into pages — escape output, use CSP, httpOnly cookies. CSRF: attacker makes the browser send a request with your cookie — use SameSite cookies or CSRF tokens.

62. How do you secure webhooks? Verify the provider's signature (HMAC) or a secret token, respond 200 quickly, process idempotently, and store the raw payload for audit.

PART 7 — MongoDB

63. SQL vs NoSQL — when to use which? SQL: structured data, relations, transactions (orders, appointments, payments). NoSQL (MongoDB): flexible schemas, documents, high write rate (chat messages, logs, scraped data).

My example: "Hospital CRM uses SQL Server because data is relational. The WhatsApp platform uses MongoDB because messages have varied payloads."

64. What is an index? Types in MongoDB? A data structure that makes queries faster (like a book index) but slows writes slightly. Types: single field, compound, multikey (arrays), text, TTL, unique, partial.

65. Compound index and the ESR rule? Order fields as **E**quality → **S**ort → **R**ange. Example: `{ to: 1, isRead: 1, timestamp: -1 }`.

66. How do you check if a query uses an index? `.explain('executionStats')` — look for `IXSCAN` (good) vs `COLLSCAN` (full scan), and compare docs examined vs returned.

67. Aggregation pipeline? A sequence of stages that transform data: `$match` → `$group` → `$sort` → `$project` → `$lookup` (join). Put `$match` first to use indexes.

```
Message.aggregate([
  { $match: { to: myNumber, isRead: false } },
  { $group: { _id: '$from', unread: { $sum: 1 } } },
  { $sort: { unread: -1 } }
]);
```

68. Embedding vs referencing? Embed when data is read together and small (address inside user). Reference when data is large, shared or grows without limit (messages of a contact).

69. Mongoose populate vs \$lookup? `populate` does a second query from the app. `$lookup` joins inside the database in one aggregation.

70. Transactions in MongoDB? Supported on replica sets with `session.startTransaction()`. Use them when several documents must change together.

71. What are Mongoose middleware (hooks)? `pre('save')` and `post('save')` functions — for example, hashing a password before save.

72. Sharding vs replication? Replication: copies of the same data for high availability (replica set). Sharding: splits data across servers for horizontal scale.

PART 8 — SQL (MySQL / PostgreSQL / SQL Server)

73. Types of joins? INNER (matching rows), LEFT (all from left + matches), RIGHT, FULL OUTER, CROSS, SELF join.

74. WHERE vs HAVING? WHERE filters rows before grouping. HAVING filters groups after GROUP BY.

75. Clustered vs non-clustered index? Clustered: table rows are physically stored in index order (one per table, often the primary key). Non-clustered: separate structure that points to rows (many per table).

76. What is normalisation? Organising tables to remove duplicate data: 1NF (atomic values), 2NF (no partial dependency), 3NF (no transitive dependency). Reporting tables are sometimes denormalised for speed.

77. ACID? Atomicity (all or nothing), Consistency (valid state), Isolation (transactions don't interfere), Durability (saved after commit).

78. How did you optimise a slow query? (92 s → 300 ms)

"I read the execution plan. The SQL Server view used OUTER APPLY, which ran a sub-query for every row. I bypassed the view, queried base tables directly with UNION ALL for the two call legs, and added indexes on the filter and join columns. Time dropped from about 92 seconds to 200–350 ms."

79. Query optimisation checklist? Read the execution plan → add indexes on WHERE/JOIN/ORDER BY columns → select only needed columns (no `SELECT *`) → avoid functions on indexed columns in WHERE → avoid N+1 queries → paginate → use proper data types → cache results.

80. What is the N+1 problem? Fetching a list (1 query) and then one query per item (N queries). Fix with a JOIN, `IN (...)`, `$lookup` or ORM eager loading.

81. Stored procedure vs function vs view? Procedure: saved SQL logic, can modify data. Function: returns a value or table, used inside queries. View: saved SELECT used like a table.

82. What is a deadlock? Two transactions each wait for a lock the other holds. Prevent by accessing tables in the same order, keeping transactions short and using the right indexes.

83. Second highest salary?

```
SELECT MAX(salary) FROM employees WHERE salary < (SELECT MAX(salary) FROM employees);  
-- or  
SELECT salary FROM (SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) r FROM employees) t  
WHERE r = 2;
```

84. Find duplicate records?

```
SELECT phone, COUNT(*) FROM patients GROUP BY phone HAVING COUNT(*) > 1;
```

85. PostgreSQL features you've used? `DISTINCT ON` (first row per group), `NOT EXISTS` for anti-joins, partial indexes, server-side cursors (`DECLARE ... FETCH 10000`) for big syncs.

86. Connection pooling — why? Opening a DB connection is slow. A pool keeps connections open and reuses them. Set a max size so the DB isn't overloaded.

PART 9 — Redis, caching and queues

87. What is Redis and where did you use it? An in-memory key-value store — very fast. I used it for caching reports and lookups, OTP storage with TTL, session state, rate limiting, and as the broker for BullMQ queues.

88. Caching strategies? - Cache-aside (most used): read cache → if miss, read DB → store in cache with TTL. - **Write-through:** write cache and DB together. - **Invalidation:** delete or version the key when data changes.

My example: "Cache keys include a version and the data-sync timestamp, so after a new sync, old cache is automatically ignored. I only cache successful, non-empty results."

89. What happens if Redis goes down?

"Every cache call is wrapped in try/catch. The app logs it and falls back to the database. Slower, but not broken."

90. Redis data types? String, Hash, List, Set, Sorted Set (leaderboards, top-viewed), Stream, plus TTL on any key.

91. Why use a message queue? To move slow or unreliable work out of the request: emails, notifications, reports, scraping. Gives retries, scaling of workers and smoothing of traffic spikes.

92. BullMQ — what do you know? Redis-based job queue for Node. Queue adds jobs; Worker processes them. Options: `attempts`, `backoff` (exponential), `concurrency`, repeatable (cron) jobs, `removeOnComplete`. I

used it for a 4-stage scraping pipeline and the ViewReports sync worker.

93. RabbitMQ vs Kafka vs BullMQ? - **BullMQ**: simple job queue on Redis, great for Node background jobs. - **RabbitMQ**: message broker with exchanges and routing; good for task distribution between services (used in TorahAnytime). - **Kafka**: distributed log for very high-throughput event streaming and replay.

94. node-cron vs a queue? node-cron is fine for simple scheduled jobs in a single process. A queue is better when you need retries, multiple workers or persistence. With multiple instances, cron can run twice — use one instance or a distributed lock.

PART 10 — AWS, serverless and cloud

95. AWS services you've used? Lambda (serverless functions), S3 (file storage), EC2 (servers), RDS (managed SQL), API Gateway (HTTP in front of Lambda), CloudWatch (logs).

96. What is serverless? Pros and cons? You write functions; AWS runs and scales them, and you pay per request. Pros: no server management, auto-scaling, cost-efficient at low traffic. Cons: cold starts, execution time limits (15 min), harder local testing, vendor lock-in.

97. What is a cold start and how do you reduce it? The first request after idle needs a new container, which adds latency. Reduce by smaller bundles, fewer dependencies, initialising DB connections outside the handler (reused), more memory, or provisioned concurrency.

My example: "In FamilyOne I optimised Lambda functions and serverless YAML configs and load tested them with JMeter."

98. How do you upload files to S3 securely? Generate a **pre-signed URL** on the backend. The client uploads directly to S3. The backend never handles the big file, and the bucket stays private.

99. EC2 vs Lambda? EC2: full server, always on, good for long-running apps and WebSockets. Lambda: event-driven, short functions, scales to zero.

PART 11 — Docker, deployment and CI/CD

100. What is Docker? Why use it? Packages an app with its dependencies into an image that runs the same everywhere. Solves "works on my machine".

101. Image vs container? An image is the template (read-only). A container is a running instance of an image.

102. Sample Dockerfile for Node:

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
EXPOSE 3000
CMD ["node", "src/server.js"]
```

Copying `package.json` first uses layer caching, so dependencies reinstall only when they change.

103. Docker Compose? Runs multiple containers together (API + MongoDB + Redis) with one `docker-compose up`.

104. Kubernetes basics? Orchestrates containers across servers. Pod (one or more containers), Deployment (desired replicas + rolling updates), Service (stable network address / load balancing), Ingress (HTTP routing), HPA (auto-scaling).

105. What is PM2? A process manager for Node: keeps the app alive, restarts on crash, cluster mode for multiple cores, log management, `max_memory_restart`.

106. What is NGINX used for? Reverse proxy in front of Node, SSL termination, load balancing, serving static files, gzip and WebSocket proxying.

107. How did you deploy on IIS?

"IIS with iisnode. I wrote `web.config` with the iisnode handler, URL rewrite rules, number of Node processes and logging. App pool set to No Managed Code and AlwaysRunning. The React build is served as a static site with an SPA rewrite to `index.html`. One gotcha: on iisnode, `process.env.PORT` is a named pipe, not a number — so never `parseInt` it."

108. What is CI/CD? CI: every push runs install, lint, tests and build automatically. CD: after merge, the app is deployed automatically. I set it up with GitHub Actions (BikeShopShippers) and Bitbucket Pipelines (Nosotros), with Sentry and Codecov.

109. Sample GitHub Actions flow? On push → checkout → setup Node → `npm ci` → lint → test → build → (on main) deploy via SSH or Docker image push.

110. Zero-downtime deployment? PM2 `reload` (restarts cluster workers one by one), rolling updates in Kubernetes, or blue-green deployment behind a load balancer.

111. How do you monitor production? Structured logs (Winston), error tracking (Sentry), health endpoints (`/health`, `/health/ready`), uptime checks and alerts.

PART 12 — System design and microservices

112. Monolith vs microservices? Monolith: one codebase and deployment — simple, good to start. Microservices: small independent services with their own data — scale and deploy separately, but more complex (network calls, monitoring, data consistency). Start monolith, split when needed.

113. How do microservices communicate? Synchronous: REST, gRPC. Asynchronous: message queues (RabbitMQ, Kafka) and events. An API gateway is the single entry point for clients.

114. How do you scale a Node.js API? Vertical (bigger server) → Horizontal (more instances behind a load balancer, PM2 cluster) → keep the app stateless (sessions in Redis/JWT) → cache → DB read replicas and indexes → queues for heavy work → CDN for static files.

115. What is a load balancer? Distributes traffic across servers (round robin, least connections). Also does health checks.

116. Rate limiter design? Count requests per user/IP in Redis with a time window (fixed window, sliding window, token bucket). Return 429 when exceeded.

117. Design a URL shortener (common question)? API: POST `/shorten` returns a short code; GET `/:code` redirects. Code: base62 of an auto-increment ID or random 7 chars. Store code → URL in DB with a unique index. Cache hot codes in Redis. Use 301/302 redirects. Track clicks asynchronously through a queue.

118. Design a notification / WhatsApp system (use your project)?

"Producer API saves the message as `queued` and pushes a job to a queue. Workers send it to the provider with retries and backoff. Provider webhooks update status by message ID (idempotent). Socket.IO pushes updates to users. Rate limits per provider. Store raw payloads for audit."

119. CAP theorem? In a distributed system with a network partition you choose Consistency or Availability. MongoDB replica sets favour consistency by default; many caches favour availability.

120. How do you handle distributed transactions? Saga pattern: a series of local transactions, each with a compensating action if a later step fails. Or the outbox pattern for reliable event publishing.

PART 13 — Real-time: WebSockets, Socket.IO, WebRTC

121. HTTP polling vs WebSocket vs SSE? Polling: client asks repeatedly (simple, wasteful). WebSocket: full two-way persistent connection (chat). SSE: server-to-client one-way stream over HTTP (progress, notifications).

122. Socket.IO features? Auto-reconnect, rooms, namespaces, fallback to polling, acknowledgements. Scale with the Redis adapter + sticky sessions.

123. What is WebRTC? (*prepare if WebRTC stays on your resume*) Peer-to-peer audio, video and data in the browser. - **Signaling**: peers exchange SDP offer/answer and ICE candidates through your server (usually WebSocket/Socket.IO). - **STUN**: tells a peer its public IP. - **TURN**: relays media when direct connection fails (strict NAT/firewall). - Media is encrypted (DTLS-SRTP). - For many participants, use an SFU (media server) or a provider like ZegoCloud/Twilio/Agora.

Safe answer: "For doctor–patient consultation, the backend creates a session for the appointment, authenticates both users, and handles signaling over WebSocket. Media flows peer-to-peer, with a TURN server as fallback."

PART 14 — Testing, TypeScript, Nest.js, Prisma

124. How do you test APIs? Unit tests for services with Jest (mock repositories), integration tests for endpoints with Supertest, Postman collections for manual testing, load tests with JMeter.

```
test('GET /health returns 200', async () => {
  const res = await request(app).get('/health');
  expect(res.statusCode).toBe(200);
});
```

125. Unit vs integration vs e2e? Unit: one function in isolation. Integration: several parts together (API + DB). E2E: the whole system like a real user.

126. Why TypeScript? Static types catch errors at compile time, give better autocomplete and make refactoring safer. I used it in AWS Lambda projects at Morpheme.

127. interface vs type in TypeScript? Both describe shapes. `interface` can be extended and merged; `type` can also describe unions and primitives.

128. What is Nest.js? A TypeScript Node framework with an Angular-like structure: modules, controllers, providers (services), dependency injection, guards (auth), pipes (validation), interceptors. Good for large structured backends.

129. What is Prisma? A type-safe ORM. Define models in `schema.prisma`, run migrations, and use the generated client: `prisma.user.findMany({ where: { active: true } })`.

130. Sequelize vs Prisma vs raw SQL? Sequelize: older, model-based ORM. Prisma: modern, type-safe. Raw SQL: full control and best for complex reports — which is why I used it for legacy SQL Server.

PART 15 — Common live coding questions (practise these)

```
// 1. Reverse a string
const reverse = s => s.split('').reverse().join('');

// 2. Remove duplicates from an array
const unique = arr => [...new Set(arr)];

// 3. Flatten a nested array
const flat = arr => arr.flat(Infinity);

// 4. Count character frequency
const freq = s => [...s].reduce((m, c) => (m[c] = (m[c] || 0) + 1, m), {});

// 5. Debounce
function debounce(fn, ms) {
  let t;
  return (...args) => { clearTimeout(t); t = setTimeout(() => fn(...args), ms); };
}

// 6. Promise with retry and exponential backoff
async function retry(fn, attempts = 3, base = 250) {
  for (let i = 1; i <= attempts; i++) {
    try { return await fn(); }
    catch (e) { if (i === attempts) throw e; await new Promise(r => setTimeout(r, base * 2 ** (i - 1))); }
  }
}

// 7. Simple Express CRUD route with validation
router.post('/users', validate(userSchema), async (req, res, next) => {
  try { const user = await userService.create(req.body); res.status(201).json(user); }
  catch (e) { next(e); }
});

// 8. Auth middleware
const auth = (req, res, next) => {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ message: 'No token' });
  try { req.user = jwt.verify(token, process.env.JWT_SECRET); next(); }
  catch { res.status(401).json({ message: 'Invalid token' }); }
};

// 9. Group array of objects by key
const groupBy = (arr, k) => arr.reduce((acc, x) => ((acc[x[k]] ||= []).push(x), acc), {});

// 10. Implement Promise.all
function promiseAll(ps) {
  return new Promise((resolve, reject) => {
    const out = []; let done = 0;
    if (!ps.length) return resolve(out);
    ps.forEach((p, i) => Promise.resolve(p).then(v => { out[i] = v; if (++done === ps.length)
    resolve(out); }, reject));
  });
}
```

If you don't know an answer

"I haven't worked with that directly in production. The closest thing I've done is [X], where I [what you did]. I'd be happy to learn it quickly."

This sounds much better than guessing.